
UNIT 2 DATA STRUCTURES IN PYTHON

Structure

- 2.0 Introduction
- 2.1 Objectives
- 2.2 Identifiers and Keywords
- 2.3 Statements and Expressions
- 2.4 Variables
- 2.5 Operators
- 2.6 Data Types
- 2.7 Data Structures
 - 2.7.1 List
 - 2.7.2 Tuple
 - 2.7.3 Dictionaries
 - 2.7.4 Set
- 2.8 Summary
- 2.9 Answer to Check your Progress
- 2.10 References/Further Readings

2.0 INTRODUCTION

Python is a general-purpose, high level programming language which very popular and being widely used by programmers. It was created by Dutch programmer Guido Van Rossum in 1991 and further developed by the Python Software Foundation. It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code. To learn to code in any language, it helps to understand its basic components. In this unit, we will learn the basic constituents of Python programming, starting with identifiers, variables, and operators. We will see how to combine them to form expressions and statements. In this unit, you will also study the data types and data structures in Python.

2.1 OBJECTIVES

After going through this unit, you will be able to:

- Use identifiers and keywords,
- Use statements and expressions,
- Explain the basic building blocks of the Python programming language,
- Use operators in Python, and
- Use various data structures available in Python.

2.2 IDENTIFIERS AND KEYWORDS

An **identifier** is a name given to a variable, function, class or module. Identifiers may be of one or more characters in the following format:

- Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore (_). Names like myIndia, Marks_1 and mca_course are all valid examples. A Python

identifier may begin with an uppercase or a lowercase letter or an underscore (_).

- An identifier cannot start with a digit but is allowed everywhere else. For example, **1plus** is invalid, but **plus1** is perfectly fine.
- Keywords (listed in Table-1) cannot be used as identifiers.
- One cannot use spaces and special symbols like !, @, #, \$, % etc. as identifiers.
- An identifier can be of any length. (However, it is preferred to keep it short and meaningful).

Some examples of valid identifiers are: Subject1_Marks, Course, Age, Sex, etc.

Keywords of Python are a list of reserved words that have predefined meaning to the Python interpreter. These are special words and cannot be used by programmers as identifiers (variables), functions, constants, or any identifier name. Using a keyword as an identifier name causes an error. Because Python is case-sensitive, keywords must be written exactly as shown in Table 1.

Table -1: List of Keywords in Python Programming Language

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Literals are the raw data values or fixed data that are used to define constants. In other words, literals represent fixed values directly in your code without requiring computation. Literals represent a specific value in a program.

Python supports several types of literals:

1. String Literals: A string literal is a sequence of characters enclosed in quotation marks. Python allows three types of quotation marks for strings: single quotes, double quotes, and triple quotes.

- **Single Quotes:** 'Hello'
- **Double Quotes:** "Python"
- **Triple Quotes (for multi-line strings or docstrings):** In Python, triple quotes (""" or ''') are a unique syntax used to define both multi-line strings and docstrings (documentation strings). It preserves

formatting, indentation, and newlines exactly as typed without requiring explicit escape characters like `\n`.

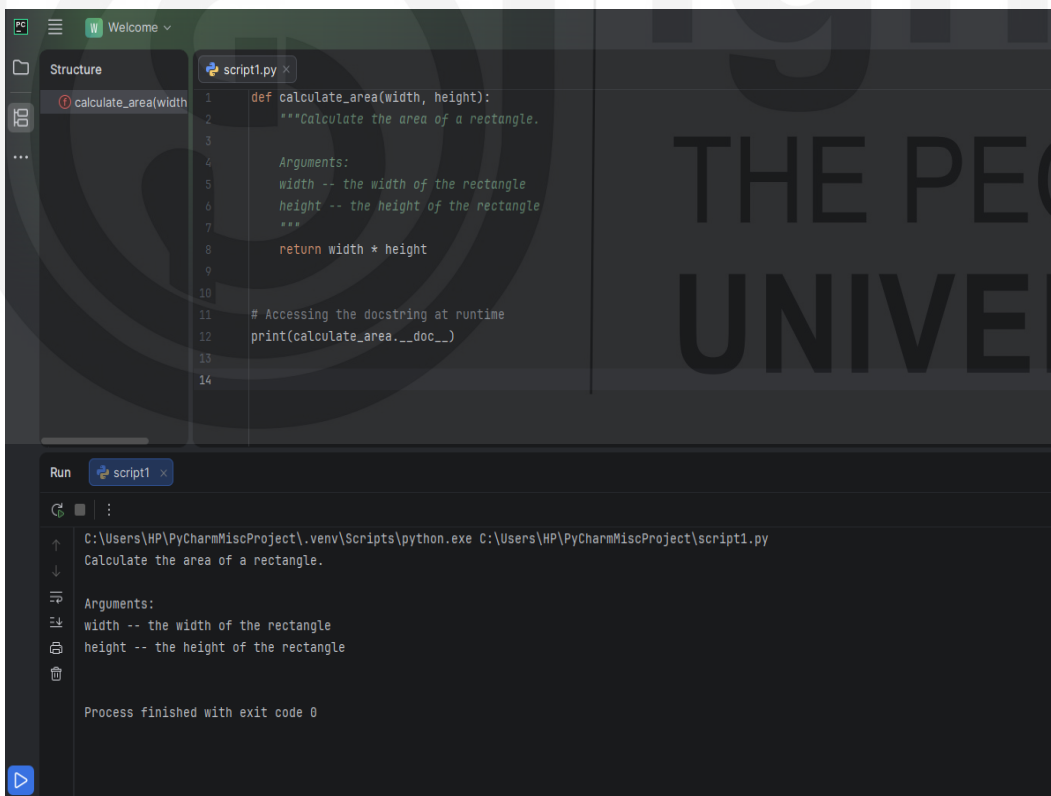
Docstrings: A docstring is a specific type of string literal used to document your code. To trigger docstring behaviour, the triple-quoted string must be the very first statement inside a function, class, or module. Python automatically binds this text to the object's `__doc__` attribute.

For example, if the following Python code:

```
def calculate_area(width, height):  
    """ Calculate the area of a rectangle.  
    Arguments:  
    width -- the width of the rectangle  
    height -- the height of the rectangle  
    """  
  
    return width * height  
# Accessing the docstring at runtime  
print(calculate_area.__doc__)
```

is executed, the output will be:

```
Arguments:  
width -- the width of the rectangle  
height -- the height of the rectangle
```



The screenshot shows a Python IDE with a file named `script1.py`. The code in the editor is as follows:

```
1 def calculate_area(width, height):  
2     """ Calculate the area of a rectangle.  
3  
4     Arguments:  
5     width -- the width of the rectangle  
6     height -- the height of the rectangle  
7     """  
8  
9     return width * height  
10  
11 # Accessing the docstring at runtime  
12 print(calculate_area.__doc__)  
13  
14
```

The IDE's Run window shows the output of the script:

```
C:\Users\HP\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\HP\PyCharmMiscProject\script1.py  
Calculate the area of a rectangle.  
  
Arguments:  
width -- the width of the rectangle  
height -- the height of the rectangle  
  
Process finished with exit code 0
```

2. Numeric Literals: In Python, numeric literals are immutable and are used to represent numbers. Python supports three distinct numeric types:

Table - 2: Numeric Literals in Python

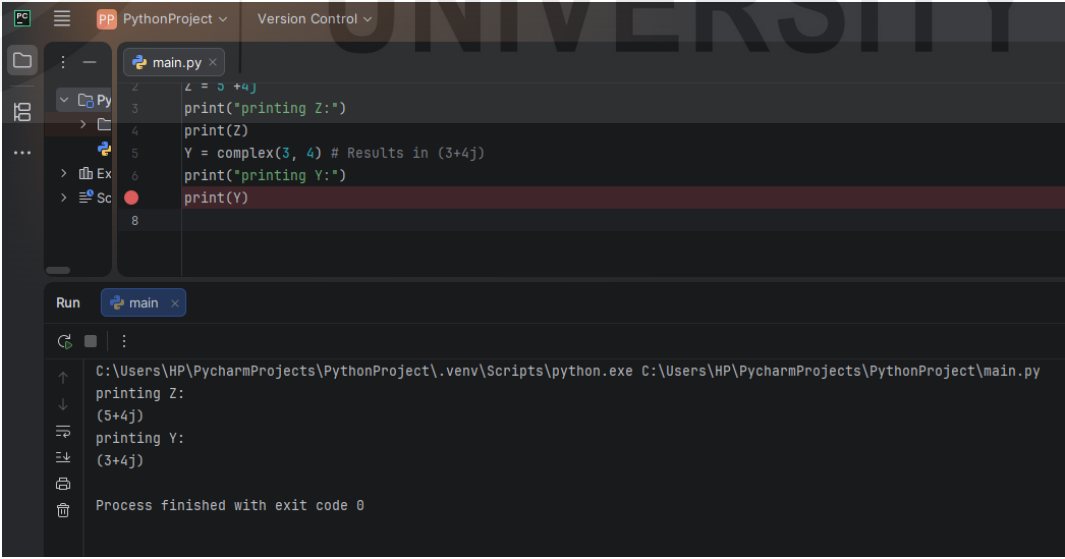
Type	Description	Examples
Integer (int)	Whole numbers, positive or negative, without a decimal point. Includes decimal, octal, hexadecimal, and binary forms.	10, -5, 0xAF (hex), 0b10101 (binary)
Float (float)	Real numbers, positive or negative, containing a decimal point or expressed in scientific notation.	10.5, -3.14, 1.2e5 (1.2 times 10 to the power of 5)
Complex (complex)	Numbers in the form $a + bj$, where a is the real part and b is the imaginary part.	$3 + 4j$

Example: Printing complex numbers

If you run the following code:

```
Z = 5 + 4j
print("printing Z:")
print(Z)
Y = complex(3, 4) # Results in (3+4j)
print("printing Y:")
print(Y)
```

As shown in the following screenshot, the `complex()` function also returns a complex number.



The screenshot shows a Python IDE with a file named `main.py`. The code in the editor is as follows:

```
1 Z = 5 + 4j
2 print("printing Z:")
3 print(Z)
4
5 Y = complex(3, 4) # Results in (3+4j)
6 print("printing Y:")
7 print(Y)
8
```

The output window at the bottom shows the execution results:

```
C:\Users\HP\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject\main.py
printing Z:
(5+4j)
printing Y:
(3+4j)
Process finished with exit code 0
```

3. Boolean Literals: A Boolean literal can have one of two values:

- True: Represents the logical truth.
- False: Represents the logical falsity.

Python has one special literal: **None**. It indicates that a variable has no value. Notably, it is not the same as 0 or an empty string. It is often used as a placeholder for a value that is yet to be provided. The following points should be noted about None:

- **It is a Singleton:** Python creates exactly one **None object** in memory during execution. All variables assigned **None point** to this identical memory address.
- **Falsy but Distinct:** In conditional statements, **None** evaluates to False. However, it does not equal 0, False, or an empty string "".
- **Implicit Return Value:** If a function does not contain an explicit return statement, Python automatically returns None.

2.3 STATEMENTS AND EXPRESSIONS

A statement is a unit of code that the Python interpreter can execute. Generally, statements fall into two kinds: expression statements and assignment statements. Note that the assignment statement produces no output.

When you type a statement in interactive mode, the interpreter executes it and displays the result, if there is one.

A script usually contains a sequence of statements. If there is more than one statement, the results appear one at a time as the statements execute.

For example, the script:

```
print(1)
x = 2
print(x)
```

produces the output:

```
1
2
```

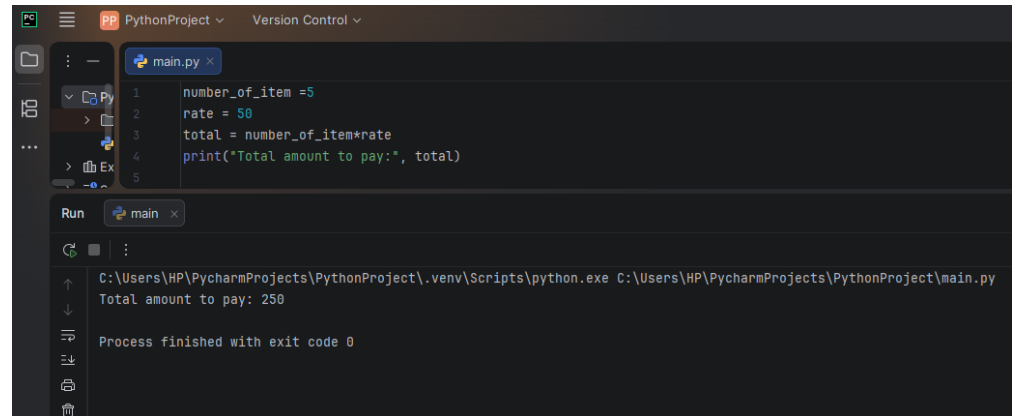
An **expression** is an arrangement of values and operators that evaluates to a new value. Expressions are statements as well. A value is the representation of some entity like a letter or a number that a program can manipulate. A single value **25** or a single variable **x** or a combination of a variable, operator and value **x + 25** are examples of expressions. In interactive mode, the interpreter evaluates an expression and displays the result instantly. For example,

```
X = 7 + 3
```

Output: 10

Another example of preparing an expression is calculating the total amount you need to pay for items you buy at a shop.

```
total = number_of_item*rate
```



```
1 number_of_item = 5
2 rate = 50
3 total = number_of_item * rate
4 print("Total amount to pay:", total)
5
```

Run main x

C:\Users\HP\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject\main.py
Total amount to pay: 250
Process finished with exit code 0

Note that in a script, an expression by itself does not show any output until you explicitly print the result.

2.4 VARIABLES

A variable is a named placeholder that holds data the program can use, assign, and modify during execution. As Python is a Dynamically Typed Language, there is no need to declare a variable explicitly by specifying whether the variable is an integer or a float or any other type. To define a new variable in Python, we simply assign a value to a name.

Variable names can be arbitrarily long. They can contain both letters and numbers, but you should follow the normal naming convention for variables. It is legal to use uppercase letters, but it may be a good idea to begin variable names with a lowercase letter. Note: variable names are case-sensitive. e.g., IGNOU and Ignou are two different variables.

The underscore character can appear in a name. It is often used in multi-word names, such as `my_name` or `marks_in_maths`. Variable names can start with an underscore character, but we generally avoid doing this unless we are writing library code for others to use.

The general format for assigning values to variables is as follows:

variable_name = expression

The equal sign (`=`), also known as the simple assignment operator, assigns values to variables. In the general format, the operand to the left of the `=` operator is the name of the variable, and the operand to the right of the `=` operator is the expression, which can be a value or any code snippet that results in a value. That value is stored in the variable on the execution of the assignment statement. The assignment operator should not be confused with the `=` used in algebra to denote equality. e.g.,

number = 100 *#integer type value is assigned to a variable number*
name = "Python" *#string type value is assigned to variable name*

In the above two statements

#integer type value is assigned to a variable number

and

#string type value is assigned to variable name

are comments. Note that comments in Python are created using the **hash character (#)**, which tells the interpreter to ignore all text following it on that line.

Python does not have multiline comments, but you can place a # at the beginning of every individual line to create multiline comments. This is a PEP 8-compliant approach to multiline comments in Python. Another way of multiline comment is to use unassigned string literals; in this approach, **you need to enclose** your comment block within triple quotes (""" or "). If a string literal is not assigned to a variable, Python allocates memory but ignores the text at runtime (effectively treating it as a comment block).

Example of multiline comment:

```
""" Try this, it is a multiline comment. It works fine as Python ignores this
block during execution. """
```

Python is dynamically typed. In Python, variables are not fixed "boxes" locked into a specific data type. Instead, variables are labels or references that point to objects in memory.

It means when we create a variable in Python:

1. It creates an object in memory that contains both the value and its data type.
2. The variable name acts as a tag attached to that object.

Here, the data type belongs to the object in memory, not the variable name. Hence, you can freely point the same variable name to an entirely different type of object whenever you like.

For example, you can assign an integer value to a variable, use it as an integer for a while and then assign a string to the variable. A new assignment overrides any previous assignments.

```
year = 2026           #an integer value is assigned to year variable
year = "twenty twenty six" # a string value is assigned to year variable
```

Python allows you to assign a single value to several variables simultaneously.

For example:

```
x = y = z = 5 #an integer value is assigned to variables x, y, z
simultaneously.
```

2.5 OPERATORS

Operators are special symbols that represent computations like addition and multiplication. The values the operator is applied to are called operands.

Python language supports a wide range of operators:

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical Operators
5. Bitwise Operators

Table - 3: List of Arithmetic Operators

Operator	Operator Name	Description	Example (try in lab) n1 = 5, n2 = 6
+	Addition operator	Adds two operands, producing their sum	R = n1 + n2 Value of R is 11
-	Subtraction operator	Subtracts the two operands, producing their difference	R = n1 - n2 Value of R is -1
*	Multiplication operator	Produces the product of the operands	R = n1 * n2 Value of R is 30
/	Division operator	Produces the quotient of its operands where the left operand is the dividend and the right operand is the divisor	R = n1 / n2 Value of R is 0.833333
%	Modulus operator	Divides the left-hand operand by the right-hand operand and returns the remainder	R = n1 % n2 Value of R is 5
**	Exponent operator	Performs exponential (power) calculation on operators	R = n1 ** n2 Value of R is 15625
//	Floor division operator	Returns the integral part of the quotient	R = n1 // n2 Value of R is 0

Table - 4: List of Assignment Operators

Operator	Operator Name	Description	Example (try in lab) (‘~’ stands for is equivalent to) n1 = 5, n2 = 6
=	Assignment	Assigns values from right-side operands to the left-side operand	m = n1 + n2 Value of m is 11
+=	Addition Assignment	Adds the value of right operand to the left operand and assigns the result to left operand	m += n1 ~ m = m + n1 when m += n1 Value of m is 16
-=	Subtraction Assignment	Subtracts the value of right operand from the left operand and assigns the result to left operand	m -= n1 ~ m = m - n1 when m -= n1 Value of m is 11
*=	Multiplication Assignment	Multiplies the value of right operand with the left operand and assigns the result to left operand	m *= n1 ~ m = m * n1 when m *= n1 Value of m is 55
/=	Division Assignment	Divides the value of right operand with the left operand and assigns the result to left operand	m /= n1 ~ m = m / n1 when m /= n1 Value of m is 11.0
**=	Exponentiation Assignment	Evaluates to the result of	m ** n1 ~ m = m **

	ion Assignment	raising the first operand to the power of the second operand	n1 when m **= n1 Value of m is 161051.0
//=	Floor Division Assignment	Produces the integral part of the quotient of its operands where the left operand is the dividend and the right operand is the divisor	m //= n1 ~ m = m // n1 when m //= n1 Value of m is 32210.0
%=	Remainder Assignment	Computes the remainder after division and assigns the value to the left operand.	m %= n1 ~ m = m % n1 when m %= n1 Value of m is 0.0

Table - 5: List of Relational Operators

Operator	Operation	Description	Example(Try in Lab) n1 = 10, n2 = 0, s1 = "Hello", s2= "World"
==	Equals to	If the values of two operands are equal, then the condition is True; otherwise, it is False.	n1 == n2 results in: False s1 == s2 results to: False
!=	Not equal to	If the values of two operands are not equal, then the condition is True; otherwise, it is False.	n1 != n2 results in: True s1 != s2 results in: False
>	Greater than	If the value of the left operand is greater than the value of the right operand, then the condition is True; otherwise, it is False.	n1 > n2 results in: True s1 > s2 results in: True
<	Less than	If the value of the left operand is less than the value of the right operand, the condition is True; otherwise, it is False.	n1 < n2 results in: False
>=	Greater than or equal to	If the value of the left operand is greater than or equal to the right operand, the condition is True; otherwise, it is False.	n1 >= n2 results in: True
<=	Less than or equal to	If the value of the left operand is less than or equal to the right operand, the condition is True; otherwise, it is False.	n1 <= n2 results in: False

Note: For strings, the characters in both strings are compared one by one based on their Unicode value. The character with the lower Unicode value is considered to be smaller.

Table - 6: List of Logical Operators

Operator	Operation	Description	Example (Try in Lab) n1 = 10, n2 = -20
And	Logical AND	If both operands are True, then the condition becomes True.	n1 == 10 and n2 == -20 results in: True
Or	Logical OR	If any of the two operands is True, then the condition becomes True.	n1 >= 10 or n2 < -20 results in: True
Not	Logical NOT	Used to reverse the logical state of its operand.	not(n1 == 20) results in: True

Table -7: Boolean Logic Truth Table

n1	n2	n1 and n2	n1 or n2	not n1
True	True	True	True	False
True	False	False	True	
False	True	False	True	True
False	False	False	False	

Table -8: List of Bitwise Operators

Operator	Operator Name	Description	Example (try in Lab) n1 = 60, n2 = 13
&	Binary AND	Result is one in each bit position for which the corresponding bits of both operands are 1s.	n1 & n2 results in: 12 (means 0000 1100)
	Binary OR	Result is one in each bit position for which the corresponding bits of either or both operands are 1s.	n1 n2 results in: 61 (means 0011 1101)
^	Binary XOR	Result is one in each bit position for which the corresponding bits of either but not both operands are 1s.	n1 ^ n2 results in: 49 (means 0011 0001)
~	Binary Ones Complement	Inverts the bits of its operand.	~n1 results in: -61 (means 1100 0011 in 2's complement form due to a signed binary number)
<<	Binary Left Shift	The left operand's value is moved left by the number of bits specified by the right operand.	n1 << 2 results in: 240 (means 1111 0000)
>>	Binary Right Shift	The left operand value is moved right by the number of bits specified by the right operand.	n1 >> 2 results in: 15 (means 0000 1111)

Table -9: Bitwise Truth Table

n1	n2	n1 & n2	n1 n2	n1 ^ n2	~n1
0	0	0	0	0	1
0	1	0	1	1	
1	0	0	1	1	0
1	1	1	1	0	

Table -10: Operator Precedence in Python

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor Division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=	Comparisons
Not	Logical NOT
And	Logical AND
Or	Logical OR

2.6 DATA TYPES

Data types specify the type of data, such as numbers and characters, that a program stores and manipulates. An important point to note is that in Python, every data type is an object, and there are no "primitive" data types as you find in languages such as C, C++ or Java. Every time we create a variable or a value, Python wraps it inside a robust container called an object. Each object in Python always contains three things:

1. **Identity:** It is a unique identifier (essentially it is a memory address, which you can check using the function `id()`).
2. **Type:** The class that defines what the object can do (which you can check using the function `type()`).
3. **Value:** The actual data stored inside the object.

Python supports the following five main built-in object types:

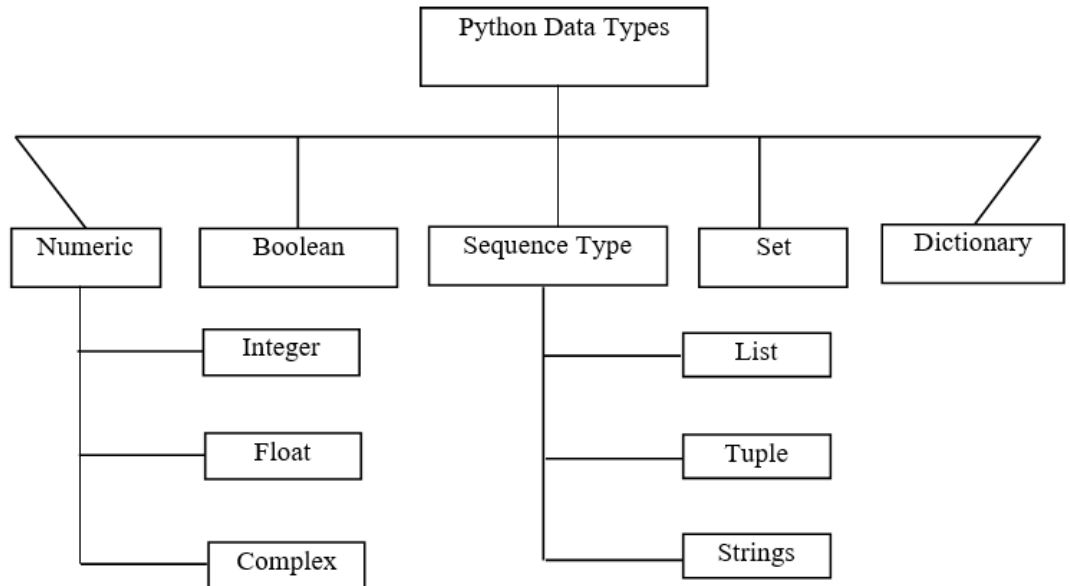


Figure 1: Python Built in Object Types

2.6.1 Numeric

Numeric data types mainly include numbers. These numbers also come in four types: Integer, Float, and Complex.

Table -11: Data Types in Python

Type	Built-in Class	Description	Examples
Integer	int	Integer numbers	-12, -3, 0, 123, 2
Float	float	Floating point numbers	-2.04, 4.0, 14.23
Complex	complex	Complex numbers	3+4j, 2-2j

2.6.2 Boolean

Boolean: In Python, Boolean contains only two values: True and False. The bool class is a direct subclass of the **int** (integer) class, and there are only two instances of this class: True (which evaluates numerically to 1) and False (which evaluates to 0).

2.6.3 String

A string is an ordered sequence of characters. It belongs to the built-in **str** class. These characters may be alphabets, digits or special characters including spaces. String values are enclosed in single quotation marks (e.g. 'hello') or in double quotation marks (e.g., "python course"). The quotes are not part of the string; they mark the string's beginning and end for the interpreter. In Python, there is no character data type; a character is a string of length one. Some of the characteristics of strings are:

- Numerical operations can not be performed on strings, even when the string contains a numeric value like str2 defined below.
- A string has a length. Use the len() built-in function to get the length.

- A string is indexable. Get a single character at a position in a string with the square bracket operator, for example `str1[4]`. Indexing always starts with 0.
- One can retrieve a slice (sub-string) of a string with a slice operation, for example `str1[4:8]`.
- Strings are immutable, which means you can not change an existing string.

Examples:

`str1 = 'Hello Friend'` #variable str1 of string type is declared

`str2 = "452"` #variable str2 of string type is declared

`len(str1)` #length of string str1

will produce: 12

`str1[4]` #to access 5th character in the string

will produce: 'o'

`str1[-1]` #negative indices can be used, which count backwards from the end of the string. [-1] yields the last letter.

will produce: 'd'

`str1[4:8]` # Operator returns the part of the string from the first index to the second index, including the first but excluding the last. So, the substring starting from the 5th character to the 8th character is extracted.

will produce: 'o Fr'

If you omit the first index (before the colon), the slice starts at the beginning of the string; if you omit the second index, the slice goes to the end of the string.

`str1[:3]`

will produce: 'hel'

`str1[3:]`

will produce: 'lo Friend'

`str1[2] = 'a'`

TypeError: 'str' object does not support item assignment.

`str2 = 'Narender'`

`str1 + str2` #for concatenating two strings, "+" operator is used

will produce: 'hello Friend Narender'

```
str1 * 3                                #for creating multiple concatenated copies of a  
                                         string, “*” operator is used
```

will produce: ‘hello FriendhelloFriendhello Friend’

2.6.4 List

A list is a sequence of items separated by commas and enclosed in square brackets []. These are similar to arrays in other programming languages, but unlike arrays, list items can be of different data types. Lists are mutable, and hence, they can be altered even after their creation. Python lists are ordered and have a definite sequence, and indexing begins at 0 (0 being the first index). Each element has a definite place in the list, which allows duplicates, with each element having its own distinct place and credibility. When describing list elements as having “credibility”, it means to say that predictability and reliability: as every element’s place is guaranteed and distinct, hence, you can trust its location and identity completely. It means a list in Python is mutable, and you can change, add, or remove its elements after it has been created without altering its unique memory identity (**id()**). It belongs to the built-in class list.

```
list1 = [5, 3.4, 'IGNOU', 'Part 3', 45]    #to create a list  
list1                                       #to print the elements of the list list1  
will produce: [5, 3.4, 'IGNOU', 'Part 3', 45]  
  
list1[0]                                   #accessing first element of the list  
will produce: 5  
  
list1[3]                                   #accessing 4th element of the list  
will produce: Part 3  
  
list1[-1]                                  #accessing last element of the list  
will produce: 45  
  
list1[-3]                                  #accessing last 3rd element of the list  
will produce: 'IGNOU'
```

2.6.5 Tuple

A tuple is similar to the list in many ways. Like lists, tuples also contain a collection of items of different data types. It belongs to the built-in **class tuple**. The items of the tuple are separated with a comma(,) and enclosed in parentheses(()). A tuple is a **read-only data structure**, as we can not modify the size and value of the items of a tuple, and it is immutable.

```
t = ('hello','world',2)                   #creating tuple t  
t                                           # printing tuple t  
will produce: ('hello', 'world', 2)  
  
t[1]                                       #accessing specific element of the tuple say second element  
will produce: 'world'
```

2.6.6 Dictionary

A dictionary in Python holds data items in **key-value pairs of items**. It belongs to the built-in **class dict**. Items in a dictionary are separated by commas and enclosed in curly brackets {}. Dictionaries permit faster access to data. Every key is separated from its value using a colon(:) sign. Keys in a dictionary are case-sensitive. Keys can not be repeated and are immutable. You can access a dictionary's key-value pairs using the key. Keys are usually strings, and values can be any data type and can be duplicated. To access a value in a dictionary, we must specify its key in square brackets[].

#Creating dictionary dict1

```
dict1 = {'Fruit':'Apple','Climate':'Cold','Price(Kg)':120}
```

```
dict1          #printing the contents of dictionary dict1
```

will produce: {'Fruit': 'Apple', 'Climate': 'Cold', 'Price(Kg)': 120}

```
dict1['Climate'] #Getting value by specifying the key
```

will produce: Cold

```
dict1.keys()    #printing all the Keys in the dictionary
```

will produce: dict_keys(['Fruit', 'Climate', 'Price(Kg)'])

```
dict1.values()      #printing all the values in the dictionary
```

will produce: dict_values(['Apple', 'Cold', 120])

2.6.7 Set

In Python, a **set** is an unordered collection of data types that is iterable, mutable and has no duplicate elements. It belongs to the built-in set class. The order of elements in a set is undefined, though it may consist of various elements. The major advantage of using a set, as opposed to a list, is its highly optimised method for checking whether a specific element is in the set.

You can create sets using the built-in set() function with an iterable object or by placing a sequence in curly braces, separated by commas. A set contains only unique elements, but at the time of set creation, multiple duplicate values can also be passed. The order of elements in a set is undefined and is unchangeable. Elements in a set need not be the same type; you can also pass mixed data types to a set.

```
set1 = set("IGNOU MCA BATCH") #creating set set1
```

```
set1    #displaying contents of set1
```

will produce: {' ', 'A', 'B', 'C', 'G', 'H', 'I', 'M', 'N', 'O', 'T', 'U'}

```
set2 = set([2, 3, 2, 'MCA', 'IGNOU', 1, 'IGNOU']) #set with the use of mixed values
```

```
set2
```

will produce: {1, 2, 3, 'IGNOU', 'MCA'}

2.6.8 None

The **None** is another special data type in Python. A **None** is frequently used to represent the absence of a value. **None** is the "null" of Python, and **NoneType** is its type. In other words, **NoneType** is the type of the **None** object (**None** belongs to the class **NoneType**).

```
money = None           #None value is assigned to variable money
```

Interestingly, some Python literature treats List, Tuple, Set, and Dictionary as additional data types, while others treat them as built-in data structures. We will put them in the category of data structures and discuss them there, but considering them as data types is also not wrong.

Check your progress - 1

Q1. What is the maximum possible length of an identifier in Python?

- a) 32 characters
- b) 64 characters
- c) 79 characters
- d) An identifier can be of any length

Q2. Which of the following can be part of an expression in Python?

- a) Values
- b) Variables
- c) Operators
- d) All of the above

Q3. What is the value of variable **a** after executing the following code?

```
a = 25  
b = 35  
a = b
```

- a) 25
- b) 35
- c) None
- d) It will raise an error

Q4. What error occurs when you execute the following Python code snippet (assuming **mango** has not been defined)?

```
apple = mango
```

- a) `SyntaxError`
- b) `NameError`
- c) `ValueError`
- d) `TypeError`

5. What is the output of `print(2 ** 3 ** 2)`?

- a) 64
- b) 128
- c) 256
- d) 512

6. What is the output of the following code?

```
a = 100  
b = 200  
print (a and b)
```

- a) 100
- b) 200
- c) True

d) False

7. 4 is 100 in binary and 11 is 1011. What is the output of the following bitwise operators?

```
a = 4
```

```
b = 11
```

```
print (a | b)
```

```
print (a >> 2)
```

a) 15, 1

b) 14, 1

c) 15, 2

d) 14, 2

8. If we convert one data type to another, then it is called

a) Type Conversion

b) Type Casting

c) Both of the above

d) None of the above

9) State the rules for naming identifiers in Python.

10) Can an expression exist inside a statement? Explain with the help of a simple program.

11) Do we need to declare variables before using them in Python?

12) What is the difference between “/ and //” operators? Explain with an example.

2.7 DATA STRUCTURES

Data structures organise, store, and manage data for efficient access and modification. Some literature on Python categorises List, Tuple, Dictionary, and Set as data types, while others categorise them as primitive data structures. In Python, List, Tuple, Dictionary, and Set are officially built-in compound data types, but structurally, they are non-primitive data structures. As you know, lists, tuples, sets, and dictionaries have internal rules governing how data is ordered, indexed, and stored; that is why they are categorised as built-in data structures.

We have already seen these structures in the previous section of this unit. Now let us discuss them in detail here:

2.7.1 List

A **list** is a container data type that acts as a dynamic array. A list is a sequence you can index, and that can grow and shrink. A few characteristics of lists are:

- A list has a (current) length – Get the length of a list with the len() function.
- A list has an order – the items in a list are ordered, with order going from left to right.
- A list is heterogeneous –different types of objects can be inserted into the same list.
- Lists are mutable, and one can extend, delete, insert or modify items in the list.

The syntax for creating a list is:

```
list_name = [item_1, item_2, item_3, ....., item_n]
```

You can store any item in a list, such as a string, number, object, another variable, or even another list. A list can contain a mix of different item types, and these types need not be homogeneous. For example, a list can mix numbers, strings, and another list. The contents of the list variable are displayed by executing the list variable name.

Examples :

```
list_name = [ ]           #empty list without any it
```

```
#when each item in the list is a string
```

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
```

```
#when each item in the list is a number
```

```
numlist = [4, 6, 7, 10, 15, 13]
```

```
#when list contains mixed items
```

```
mixed_list = ['cat', 87.23, 65, [9, 8, 1]]
```

2.7.1.1 Indexing and Slicing in Lists

As an ordered sequence of elements, each item in a list can be accessed individually through indexing. The expression inside the bracket is called the index. Lists use square brackets [] to access individual items, with the first item at index 0, the second item at index 1 and so on. The index provided within the square brackets indicates the value being accessed.

The syntax for accessing an item in a list is:

list_name [index]

where index should always be an integer value and indicates the item to be selected.

if

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
```

then

```
stringlist[0]          #to access the first item in stringlist
```

will produce: 'mahesh'

and

in

```
numlist = [4, 6, 7, 10, 15, 13]
```

if you access

```
numlist[4]             #to access fifth item in the numlist
```

it will produce: 15

and

```
numlist[6]             #if index value is more than the number of items in the list, it  
will raise an error
```

If you try to access numlist[6] output would be :

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

```
numlist[6]
```

IndexError : List index out of range

Here, in numlist, index numbers range from 0 to 5 as it contains 6 items.

In addition to positive index numbers, you can also access items in a list with a negative index number by counting backwards from the end of the list, starting at -1. It is useful if the list is long and we want to locate an item towards the end of the list.

For example, if you access

```
stringlist[-2]         #to access the second list item from the list
```

it will produce: 'vishnu'

The slice operator also works on lists:

```
numlist[1:3]
```

will produce: [6, 7]

```
mixed_list[:3]
```

will produce: ['cat', 87.23, 65]

`mixed_list[2:]`

will produce: [65, [9, 8, 1]]

`numlist[:]`

will produce: [4, 6, 7, 10, 15, 13]

If you omit the first index, the slice starts at the beginning; if you omit the second index, the slice goes to the end. So, if both indices are omitted, the slice copies the whole list.

2.7.1.2 List Operations

The + operator concatenates lists:

```
a = [1, 2, 3]
```

```
b = [4, 5, 6]
```

```
then a + b
```

```
will be : [1, 2, 3, 4, 5, 6]
```

Similarly, the * operator repeats a list given a number of times:

```
c = ['pass']
```

```
c * 3
```

```
will be : ['pass', 'pass', 'pass']
```

== operator is used to compare the two lists.

```
a == b
```

```
will be: False
```

in and not in membership operators are used to check for the presence of an item in the list:

2 in a will give True and 4 not in [a] will also give True

2.7.1.3 The list() function

The built-in list () function creates a list. The syntax for list() function is,

```
list([sequence])
```

where the sequence can be a string, tuple or list itself. If you don't specify the optional sequence, it creates an empty list.

```
greet = "How are you doing?"           #string declaration
```

```
greet
```

will produce: 'How are you doing?'

```
str_to_list = list(greet)               #string converted to list using list()
```

```
str_to_list
```

will produce: ['H','o','w',' ','a','r','e',' ','y','o','u',' ','d','o','i','n','g','?']

```
friend = "nidhi"                #string declaration
```

```
str_to_list + friend            #list concatenated with string
```

TypeError: can only concatenate list (not "str") to list

```
str_to_list + list(friend)      #list concatenated with list
```

will produce : ['H','o','w',' ','a','r','e',' ','y','o','u',' ','d','o','i','n','g','?','n','i','d','h','i']

2.7.1.4 Modifying items in Lists

Lists are mutable because you can modify list items after a list is created. You can also modify a list by replacing an older item with a newer one, without assigning the list to a completely new variable.

```
stringlist[1] = "umesh"
```

and

```
stringlist
```

will produce: ['mahesh', 'umesh', 'suresh', 'vishnu', 'ganesh']

When an existing list variable is assigned to a new variable, an assignment (=) on lists does not make a new copy. Instead, assignment makes both variable names point to the same list in memory. That is why any change in one list will also reflect in the other.

```
listofstrings = stringlist
```

```
listofstrings[1] = "ramesh"
```

```
listofstrings
```

will produce: ['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']

and

```
stringlist
```

will produce: ['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']

2.7.1.5 Traversing a list

The most common way to traverse the elements of a list is with a for loop.

for names in stringlist :

```
print(names)
```

will produce:

```
mahesh
```

```
ramesh
```

suresh

vishnu

ganesh

This works well if one needs to read the elements of the list. But if you want to write or update the elements, you need the indices. For this, one has to combine the range () function and len() function:

```
for i in range(len(numlist)) :
```

```
    numlist[i] = numlist[i] * 2
```

This loop traverses the list and updates each element. The len () command returns the number of elements in the list. The range () command returns a list of indices from 0 to n-1, where n is the list length. In each loop iteration, the variable i gets the index of the next element. The assignment statement in the body uses i to read the old value of the element and to assign the new value.

```
numlist
```

will produce: [8, 12, 14, 20, 30, 26]

2.7.1.6 List Methods

Python provides methods that operate on lists.

For example,

- **append** adds a new element to the end of a list:

```
numlist.append(50)
```

now,

```
numlist
```

will produce: [8, 12, 14, 20, 30, 26, 50]

- **extend** takes a list as an argument and appends all of the elements:

```
nlist = [25, 35, 45]
```

when we do the following:

```
numlist.extend(nlist)
```

```
numlist
```

will produce: [8, 12, 14, 20, 30, 26, 50, 25, 35, 45]

- **sort** arranges the elements of the list from low to high:

```
numlist.sort()
```

now,

```
numlist
```

will produce: [8, 12, 14, 20, 25, 26, 30, 35, 45, 50]

- **pop** is used to delete elements from a list if the index of the element is known:

```
numlist.pop(5)
```

will produce: 26

and now,

```
numlist
```

will produce: [8, 12, 14, 20, 25, 30, 35, 45, 50]

pop modifies the list and returns the removed element.

- If the removed element is not required, then one can use the **del** operator:

```
del numlist[0]
```

then,

```
numlist
```

will produce: [12, 14, 20, 25, 30, 35, 45, 50]

- To remove more than one element, **del** can be used with a slice index

```
del numlist[0:2]
```

now,

```
numlist
```

will produce: [20, 25, 30, 35, 45, 50]

- If an element from the list is known which needs to be removed and not the index, **remove** can be used:

```
numlist.remove(30)
```

now,

```
numlist
```

will produce: [20, 25, 35, 45, 50]

The return value for remove is None.

To get a list of all methods associated with the list, pass the list to **dir()**.

dir(list)

Table-12: Various List Methods

List Methods	Syntax	Description
append()	list.append(item)	The append() method adds a single item to the end of the list. This method does not return a new list, and it just modifies the original.
count()	list.count(item)	The count() method counts the number of times the item has occurred in the list and returns it.
insert()	list.insert(index, item)	The insert() method inserts the item at the given index, shifting items to the right.
extend()	list.extend(list2)	The extend() method adds the items in list2 to the end of the list.
index()	list.index(item)	The index() method searches for the given item from the start of the list and returns its index. If the value appears more than once, it returns the index of the first occurrence. If the item is not present in the list, then ValueError is thrown by this method.
remove()	list.remove(item)	The remove() method searches for the first instance of the given item in the list and removes it. If the item is not present in the list, this method throws a ValueError.
sort()	list.sort()	The sort() method sorts the items in the list in place. This method modifies the original list, and it does not return a new list.
reverse()	list.reverse()	The reverse() method reverses the items in place in the list. This method modifies the original list, and it does not return a new list.
pop()	list.pop([index])	The pop() method removes and returns the item at the given index. If you omit the index, it returns the rightmost item.

Note: Replace the term “list” mentioned in the syntax with your actual list name in your code.

2.7.1.7 Built-In Functions Used on Lists

Many built-in functions accept a list as an argument.

Table -13: Built-In Functions Used on Lists

Built-In Functions	Description
len()	The len() function returns the number of items in a list.
sum()	The sum() function returns the sum of numbers in the list.

any()	The any() function returns True if any of the Boolean values in the list is True.
all()	The all() function returns True if all the Boolean values in the list are True, else returns False.
sorted()	The sorted() function returns a modified copy of the list while leaving the original list untouched.

```
len(stringlist)
```

```
will produce: 5
```

```
sum(numlist)
```

```
will produce:175
```

```
max(numlist)
```

```
will produce:50
```

```
min(numlist)
```

```
will produce:20
```

```
any([1, 1, 0, 0, 1, 0])
```

```
will produce:True
```

```
all([1, 1, 1, 1])
```

```
will produce:True
```

```
sorted_stringlist = sorted(stringlist)
```

```
sorted_stringlist
```

```
will produce: ['ganesh', 'mahesh', 'ramesh', 'suresh', 'vishnu']
```

2.7.1.8 Nested List

A list inside another list is called a **nested list**, and you can get the behaviour of nested lists in Python by storing lists within the elements of another list. The syntax for nested lists is:

```
nested_list_name = [[item_1, item_2, item_3],
                    [item_4, item_5, item_6],
                    [item_7, item_8, item_9]]
```

Each inner list is separated by a comma. You can traverse the items in nested lists using a for loop.

```
asia = [["India", "Japan", "Korea"],
        ["Srilanka", "Myanmar", "Thailand"],
        ["Cambodia", "Vietnam", "Israel"]]
```

here,
asia[0]
will produce: ['India', 'Japan', 'Korea']
asia[0][1]
will produce: 'Japan'
asia[1][2]
will produce: 'Thailand'

Python Program to find the Transpose of a Matrix:

```
1. matrix = [[10, 20],  
2.          [30, 40],  
3.          [50, 60]]  
4. matrix_transpose = [[0, 0, 0],  
5.                     [0, 0, 0]]  
6. for rows in range(len(matrix)):  
7.     for columns in range(len(matrix[0])):  
8.         matrix_transpose[columns][rows] = matrix[rows][columns]  
9. print("Transposed Matrix is")  
10. for items in matrix_transpose:  
11.     print(items)
```

Screen shot of execution of above Program :

```
main.py x  
1 matrix = [[10, 20], [30, 40], [50, 60]]  
2 matrix_transpose = [[0, 0, 0], [0, 0, 0]]  
3 for rows in range(len(matrix)):  
4     for columns in range(len(matrix[0])):  
5         matrix_transpose[columns][rows] = matrix[rows][columns]  
6 print("Transposed Matrix is:")  
7 for items in matrix_transpose:  
8     print(items)  
9  
main x  
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py  
Transposed Matrix is:  
[10, 30, 50]  
[20, 40, 60]  
Process finished with exit code 0
```

2.7.2 Tuple

In mathematics, a tuple is a finite ordered list (sequence) of elements. A **tuple** is defined as a data structure that comprises an ordered, finite sequence of immutable, heterogeneous elements that are of fixed sizes. Often, we may want to return more than one value from a function. Tuples solve this problem. They can also be used to pass multiple values through one function parameter.

2.7.2.1 Creating Tuples

A tuple is a finite ordered list of values of possibly different types which is used to bundle related values together without having to create a specific type to hold them. Tuples are immutable. Once created, you cannot change a tuple's values. A tuple is defined by putting a comma-separated list of values inside parentheses. Each value inside a tuple is called an item. The syntax for creating tuples is:

```
tuple_name = (item_1, item_2, item_3, ....., item_n)
```

Syntactically, a tuple is a comma-separated list of values:

```
t = 'a', 'b', 'c', 'd', 'e'
```

Although it is not necessary, it is common to enclose tuples in parentheses to help us quickly identify tuples when Python code is looked at:

```
t = ('a', 'b', 'c', 'd', 'e')
```

The comma forms a tuple, so the commas matter, not the parentheses.

To create a tuple with a single element, one must include the final comma:

```
t1 = ('a',)
```

here,

```
type(t1)
```

will produce: tuple

In Python, the built-in data type for a tuple type is **tuple**. It is worth noting that without the comma, Python treats ('a') as an expression with a string in parentheses that evaluates to a string:

For example,

```
t2 = ('a')
```

and the `type(t2)`

will produce: str

One can create an empty tuple without any values. For that, the syntax is:

```
tuple_name = ()
```

```
empty_tuple = ()
```

```
empty_tuple
```

will produce: ()

One can store any type of type string, number, object, another variable, and even another tuple itself. One can have a mix of different types of items in tuples, and they need not be homogeneous.

if,

```
socis = ('mca', '6 sem', '3-6 yrs', 54000)
```

and,

```
ignou = ('soe', 'soh', socis, 'soms')
```

then,

```
ignou
```

will have: ('soe', 'soh', ('mca', '6 sem', '3 – 6 yrs', 54000), 'soms')

2.7.2.2 Basic Tuple Operations

Most list operators also work on tuples. The bracket operator indexes an element:

When,

```
t = ('a', 'b', 'c', 'd', 'e')
```

```
t[0]
```

will produce: 'a'

And the slice operator selects a range of elements.

```
t[1:3]
```

will produce: ('b', 'c')

But if you try to modify one of the elements of the tuples, you will get an error:

```
t[0] = 'A'
```

TypeError: 'tuple' object does not support item assignment

We can not modify the elements of a tuple, but can replace one tuple with another:

```
t = ('A',) + t[1:]
```

now,

```
t
```

will have: ('A', 'b', 'c', 'd', 'e')

Here, + operator is used to concatenate two tuples together. Similarly, * operator can be used to repeat a sequence of tuple items.

```
t * 2
```

will produce: ('A', 'b', 'c', 'd', 'e', 'A', 'b', 'c', 'd', 'e')

in and **not in** membership operators are used to check for the presence of an item in a tuple.

Comparison operators like <, <=, >, >=, == and != are used to compare tuples.

Python starts by comparing the first element from each sequence. If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered even if they are really big.

```
(0, 1, 2) < (0, 3, 4)
```

will produce: True

and

```
(0, 1, 500000) < (0, 3, 4)
```

will produce: True

2.7.2.3 The tuple() Function

The built-in tuple() function is used to create a tuple. The syntax for the tuple() function is:

```
tuple([sequence])
```

where sequence can be a number, string or tuple itself. If the optional sequence is not specified, then an empty tuple is created.

```
t3 = tuple()
```

t3

will produce: ()

If the argument is a sequence (string, list or tuple), the result of the call to tuple is a tuple with the elements of the sequence:

```
t3 = tuple('IGNOU')
```

then,

t3

will produce: ('I', 'G', 'N', 'O', 'U')

```
t4 = (1, 2, 3, 4)
```

```
nested_t = (t3, t4)
```

then,

nested_t

will produce: (('I', 'G', 'N', 'O', 'U'), (1, 2, 3, 4))

2.7.2.4 Built-In Functions Used on Tuples

There are many built-in functions, as listed in Table -14, for which a tuple can be passed as an argument.

Table -14: Built-In Functions Used on Tuples

Built-In Functions	Description
len()	The len() function returns the number of items in a tuple.
sum()	The sum() function returns the sum of numbers in the tuple.
sorted()	The sorted() function returns a sorted copy of the tuple as a list while leaving the original tuple untouched.

```
len(t3)
```

will produce: 5

```
sum(t4)
```

will produce: 10

```
t5 = sorted(t3)
```

then,

```
t5
```

will produce: ['G', 'I', 'N', 'O', 'U']

2.7.2.5 Tuple Assignment

One of the unique syntactic features of the Python language is the ability to have a tuple on the left side of an assignment statement. This lets you assign multiple variables at once when the left side is a sequence.

In this example, we have a two-element list (a sequence) and assign its first and second elements to the variables x and y in a single statement.

```
m = ['good', 'luck']
```

```
x, y = m
```

```
x
```

will produce: 'good'

```
y
```

will produce: 'luck'

Python roughly translates the tuple assignment syntax to be the following:

```
m = ['good', 'luck']
```

```
x = m[0]
```

```
y = m[1]
```

Stylistically, when we use a tuple on the left side of the assignment statement, we omit the parentheses, but the following is an equally valid syntax:

```
(x, y) = m
```

2.7.2.6 Relation between Tuples and Lists

Though tuples may seem similar to lists, they are often used in different situations and for different purposes. Tuples are immutable and usually contain a heterogeneous sequence of elements accessed via unpacking or

indexing. Lists are mutable, and you access their items via indexing. You cannot add, remove, or replace items in a tuple.

If an item within a tuple is mutable, then you can change it. Consider the presence of a list as an item in a tuple; then any changes to the list get reflected in the overall items in the tuple.

```
if
ignou = ['soe', 'soms', 'socis']
univ = ('du', 'ggsipu', 'jnu', ignou)
then,
univ
will have: ('du', 'ggsipu', 'jnu', ['soe', 'soms', 'socis'])
and after
univ[3].append('soss')
univ
will have: ('du', 'ggsipu', 'jnu', ['soe', 'soms', 'socis', 'soss'])
```

2.7.2.7 Tuple Methods

To get a list of all the methods associated with the tuple, pass the tuple to `dir()`.

Methods associated with tuples are listed in Table - 15

Table -15: Tuple Methods

Tuple Methods	Syntax	Description
<code>count()</code>	<code>tuple_name.count(item)</code>	The <code>count()</code> method counts the number of times the item has occurred in the tuple and returns it.
<code>index()</code>	<code>tuple_name.index(item)</code>	The <code>index()</code> method searches for the given item from the start of the tuple and returns its index. If the value appears more than once, it returns the index of the first occurrence. If the item is not present in the tuple, then <code>ValueError</code> is thrown by this method.

Note: You may replace the word “`tuple_name`” mentioned in the syntax with your actual tuple name in the code.

Consider followings:

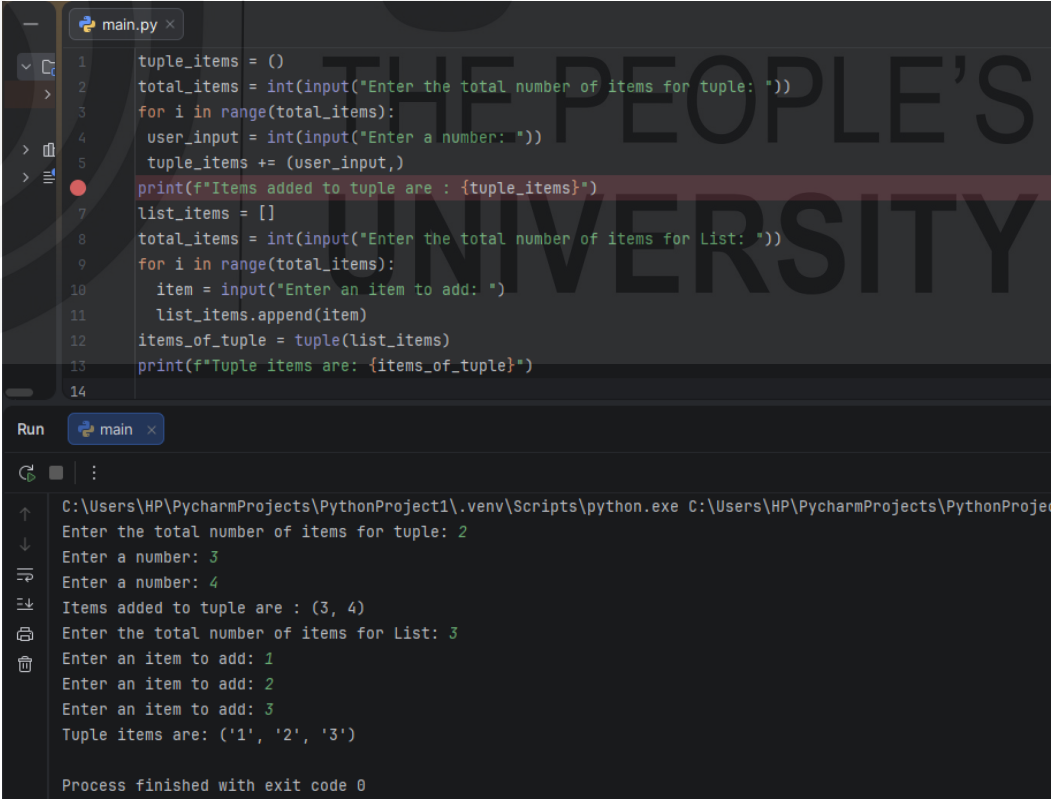
```
channels = ("dd", "star", "sony", "zee", "dd", "sab")
channels.count("dd")
```

will result: 2
channels.index("dd")
will give: 0
channels.index("zee")
will give: 3
and
channels.count("sab")
will give: 1

Program for Creating/Populating Tuple and List with User-Entered Items

```
tuple_items = ()
total_items = int(input("Enter the total number of items for tuple: "))
for i in range(total_items):
    user_input = int(input("Enter a number: "))
    tuple_items += (user_input,)
print(f"Items added to tuple are : {tuple_items}")
list_items = []
total_items = int(input("Enter the total number of items for List: "))
for i in range(total_items):
    item = input("Enter an item to add: ")
    list_items.append(item)
items_of_tuple = tuple(list_items)
print(f"Tuple items are: {items_of_tuple}")
```

Screenshot of execution of the above program:



The screenshot shows a Python IDE with a file named 'main.py' open. The code in the editor is as follows:

```
1 tuple_items = ()
2 total_items = int(input("Enter the total number of items for tuple: "))
3 for i in range(total_items):
4     user_input = int(input("Enter a number: "))
5     tuple_items += (user_input,)
6     print(f"Items added to tuple are : {tuple_items}")
7 list_items = []
8 total_items = int(input("Enter the total number of items for List: "))
9 for i in range(total_items):
10     item = input("Enter an item to add: ")
11     list_items.append(item)
12 items_of_tuple = tuple(list_items)
13 print(f"Tuple items are: {items_of_tuple}")
14
```

The output console shows the following execution:

```
Enter the total number of items for tuple: 2
Enter a number: 3
Enter a number: 4
Items added to tuple are : (3, 4)
Enter the total number of items for List: 3
Enter an item to add: 1
Enter an item to add: 2
Enter an item to add: 3
Tuple items are: ('1', '2', '3')
Process finished with exit code 0
```

Items are inserted into the tuple using two methods: using the continuous concatenation += operator and by converting list items to tuple items. In the code, tuple_items is of tuple type. In both methods, the code specifies the total number of items to insert into the tuple beforehand. Based on this number, the loop is iterated using the range() function. In the first method,

the user-entered items are continuously concatenated to the tuple using the += operator. Tuples are immutable and should not be changed. During each iteration, each original_tuple is replaced by original_tuple + (new_element), thus creating a new tuple. Notice a comma after new_element. In the second method, we create a list. For each iteration, the user-entered value is appended to the list_variable. This list is then converted

Program to Swap Two Numbers without Using Intermediate/Temporary

```
a = int(input("Enter a value for the first number:"))
b = int(input("Enter a value for the second number:"))
b,a = a,b
print("After Swapping")
print(f"Value for first number: {a}")
print(f"Value for second number: {b}")
```

Screenshot of execution of the program:

```
main.py
1 a = int(input("Enter a value for the first number:"))
2 b = int(input("Enter a value for the second number:"))
3 b,a = a,b
4 print("After Swapping")
5 print(f"Value for first number: {a}")
6 print(f"Value for second number: {b}")
7

Run
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
Enter a value for the first number:5
Enter a value for the second number:8
After Swapping
Value for first number: 8
Value for second number: 5
Process finished with exit code 0
```

The contents of variables a and b are reversed. The tuple variables are on the left side of the assignment operator, and the tuple values are on the right side. The number of variables on the left and the number of values on the right must match. Each value is assigned to its respective variable.

Check your progress – 2

- Which of the following statements creates an empty list in Python?
 - l = []
 - l = list()
 - Both a and b
 - Neither a nor b
- What error is raised in Python when attempting to access a list element using an invalid index position?
 - NameError: list is not defined
 - IndexError: list index out of range
 - BoundsError: list index out of bounds
 - No error (returns None)
- Given tupl = (5, 2, 7, 0, 3), which operation executes without error?
 - tupl[1] = 2
 - tupl.append(2)
 - tupl1 = tupl + tupl
 - tupl.sort()

4) Why is a list said to be mutable?

5) Explain the use of the “in” and “not in” operators in a list?

6) Why can tuples contain mutable items?

7) Show how to initialise a list of tuples.

2.7.3 Dictionaries

In the real world, you have seen your contact list on your phone. It is practically impossible to memorise everyone’s mobile number. In the contact list, you store the person's name as well as their number. This lets you identify a mobile number by a person’s name. One can think of a person’s name as the key that retrieves his mobile number, which is the value associated with the key. So, a dictionary can be thought of as:

- an unordered collection of key-value pairs, with the requirement that the keys are unique within a dictionary.
- as a mapping between a set of indices (which are called keys) and a set of values. Each key maps to a value. The association of a key and a value is called a key-value pair or sometimes an item.
- has a length, specifically the number of key-value pairs.
- provides fast lookup by key.

Dictionary keys must be immutable object types and are case-sensitive. Keys can be strings or numbers. But lists can not be used as keys. A value in the dictionary can be of any data type, including string, number, list or dictionary itself.

Dictionaries are constructed using curly braces {}, wherein a list of key: value pairs gets separated by commas. There is a colon(:) separating each of these **key: value** pairs, where the words to the left of the colon operator are the keys and the words to the right of the colon operator are the values.

The syntax for creating a dictionary is :

```
dictionary_name = {key_1:value_1, key_2:value_2, key_3:value_3, ..... ,  
key_n : value_n}
```

Example:

```
eng2sp = {'one' : 'uno', 'two' : 'dos', 'three' : 'tres'}
```

```
print(eng2sp)
```

will display: {'one' : 'uno', 'two' : 'dos', 'three' : 'tres'}

With Python 3.6 onwards, dictionary output shows ordered key:value pairs. Here, ordered means “insertion ordered”, i.e, dictionaries remember the order in which the **key:value** pairs were inserted. The elements of a dictionary are never indexed with integer indices. Instead, the keys are used to look up the corresponding values:

```
print(eng2sp['two'])
```

will display: dos

The key 'two' always maps to the value 'dos' so the order of the items does not matter. If the key is not in the dictionary, one get an exception:

```
print( eng2sp['four'])
```

will display: KeyError: 'four'

Slicing is not allowed in dictionaries because they are not ordered like lists.

2.7.3.1 Built-in Functions Used on Dictionaries

There are many built-in functions for which a dictionary can be passed as an argument. The main operations on a dictionary are storing a value with some key and extracting the value for a given key.

Table – 16: Built-In Functions Used on Dictionaries

Built-in Functions	Description
len()	The len() function returns the number of items (key:value pairs) in a dictionary.
all()	The all() function returns the Boolean value True if all the keys in the dictionary are True; otherwise, it returns False.
any()	The any() function returns the Boolean value True if any of the keys in the dictionary is True; otherwise, it returns False.
sorted()	The sorted() function by default returns a list of items, which are sorted based on dictionary keys.

```
print( len(eng2sp))
```

will display: 3

len() function can be used to find the number of **key:value** pairs in the dictionary eng2sp. In Python, any non-zero integer value is True, and zero is interpreted as False. With all() function, if all the keys are Boolean True values, then the output is True else it is False.

For **any()** function, if any one of the keys is True then it results in a True Boolean value else False Boolean.

```
dict_func = {0 : True, 1 : False, 4 : True}
```

```
print(all(dict_func))  
  
will display: False  
  
and  
  
print( any(dict_func))  
  
will display: True
```

The **sorted()** function returns the sorted list of keys by default in ascending order without modifying the original key:value pairs. For list of keys sorted in descending order by passing the second argument as `reverse = True`.

```
sorted(eng2sp)  
  
['one', 'three', 'two']  
  
print(sorted(eng2sp, reverse= False))  
  
will display: ['one', , 'three', 'two']
```

The **in** operator works on dictionaries; it tells whether something appears as a key in the dictionary (appearing as a value is not good enough).

```
'one' in eng2sp  
  
will result: True  
  
and  
  
'uno' in eng2sp  
  
will result: False
```

The **in** operator uses different algorithms for lists and dictionaries. For lists, it uses a linear search algorithm. As a list gets longer, search time increases in direct proportion to the list's length. For dictionaries, Python uses an algorithm called a hash table, so the **in** operator takes about the same amount of time no matter how many items there are in a dictionary.

2.7.3.2 Dictionary Methods

Various methods associated with dictionary are given in Table -17.

Table -17: Various Dictionary Methods

Dictionary Methods	Syntax	Description
clear()	dictionary_name.clear()	The clear() method removes all the key:value pairs from the dictionary.
fromkeys()	dictionary_name. fromkeys(seq[, value])	The fromkeys() method creates a new dictionary from the given sequence of elements with a value provided by the user.
get()	dictionary_name. get(key[, default])	The get() method returns the value associated with the specified key in the dictionary. If the

		key is not present, then it returns the default value. If default is not given, it defaults to None, so that this method never raises a KeyError.
items()	dictionary_name.items()	The items() method returns a new view of the dictionary's key and value pairs as tuples.
keys()	dictionary_name.keys()	The keys() method returns a new view consisting of all the keys in the dictionary.
pop()	dictionary_name. pop(key [, default])	The pop() method removes the key from the dictionary and returns its value. If the key is not present, then it returns the default value. If the default is not given and the key is not in the dictionary, then it results in KeyError.
popitem()	dictionary_name. popitem ()	The popitem() method removes and returns an arbitrary (key, value) tuple pair from the dictionary. If the dictionary is empty, then calling popitem() results in KeyError.
setdefault()	dictionary_name. setdefault(key[, default])	The setdefault() method returns a value for the key present in the dictionary. If the key is not present, then insert the key into the dictionary with a default value and return the default value. If the key is present, the default is None, so this method never raises a KeyError.
update()	dictionary_name. update([other])	The update() method updates the dictionary with the key:value pairs from another dictionary object, and it returns None.
values()	dictionary_name.values()	The values() method returns a new view consisting of all the values in the dictionary.

Note: Replace the word “dictionary_name” mentioned in the syntax with the actual dictionary name.

Example:

```
million_dollar = {"sanju" : 2018, "tiger zindahai" : 2017, "baahubali 2" : 2017, "dangal" : 2016, "bajrangibhaijaan" : 2015}
```

```
bolwd_million_dollar = million_dollar.fromkeys(million_dollar, "1,00,00,000")
```

```
print(bolwd_million_dollar)
```

will display:

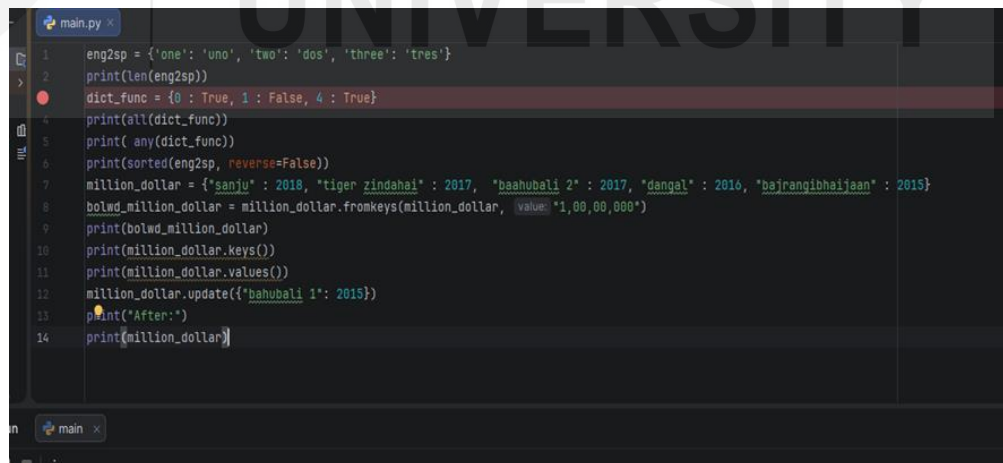
```
{'sanju': '1,00,00,000', 'tiger zindahai': '1,00,00,000',  
 'baahubali 2': '1,00,00,000', 'dangal': '1,00,00,000',
```

```
'bajrangibhaijaan': '1,00,00,000'}  
  
print( million_dollar.get("baahubali 1"))  
will display:None  
  
print(million_dollar.get("baahubali 2"))  
will display: 2017  
  
print( million_dollar.keys())  
will display: dict_keys(['sanju', 'tiger zindahai', 'baahubali 2', 'dangal',  
'bajrangibhaijaan'])  
  
print(million_dollar.values())  
will display: dict_values([2018, 2017, 2017, 2016, 2015])  
  
print(million_dollar.items())  
will display: dict_items([('sanju', 2018), ('tiger zindahai', 2017),  
( 'baahubali 2', 2017), ('dangal', 2016), ('bajrangibhaijaan', 2015)])
```

When we update :

```
million_dollar.update({"bahubali 1" : 2015})  
print(million_dollar)  
will display: {'sanju': 2018, 'tiger zindahai': 2017,  
'baahubali 2': 2017, 'dangal': 2016, 'bajrangibhaijaan': 2015,  
'bahubali 1': 2015}
```

Screenshot of execution of the above functions:



```
main.py  
1 eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}  
2 print(len(eng2sp))  
3 dict_func = {0 : True, 1 : False, 4 : True}  
4 print(all(dict_func))  
5 print(any(dict_func))  
6 print(sorted(eng2sp, reverse=False))  
7 million_dollar = {'sanju' : 2018, 'tiger zindahai' : 2017, 'baahubali 2' : 2017, 'dangal' : 2016, 'bajrangibhaijaan' : 2015}  
8 bold_million_dollar = million_dollar.fromkeys(million_dollar, value="1,00,00,000")  
9 print(bold_million_dollar)  
10 print(million_dollar.keys())  
11 print(million_dollar.values())  
12 million_dollar.update({'bahubali 1': 2015})  
13 print("After:")  
14 print(million_dollar)
```

```

C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
3
False
True
['one', 'three', 'two']
{'sanju': '1,00,00,000', 'tiger zindahai': '1,00,00,000', 'bahubali 2': '1,00,00,000', 'dangal': '1,00,00,000', 'bajrangibhaijaan': '1,00,00,000'}
dict_keys(['sanju', 'tiger zindahai', 'bahubali 2', 'dangal', 'bajrangibhaijaan'])
dict_values(['1,00,00,000', '1,00,00,000', '1,00,00,000', '1,00,00,000', '1,00,00,000'])
After:
{'sanju': 2018, 'tiger zindahai': 2017, 'bahubali 2': 2017, 'dangal': 2016, 'bajrangibhaijaan': 2015, 'bahubali 1': 2015}
Process finished with exit code 0

```

One common way to populate dictionaries is to start with an empty dictionary {}, then use the update() method to assign a value to a key using the assignment operator. If the key does not exist, then the key:value pairs will be created automatically and added to the dictionary.

```
states = {}
```

```
states.update({"Haryana": "Chandigarh"})
```

```
states.update({"Bihar": "Patna"})
```

```
states.update({"west bengal" : "kolkata"})
```

```
print( states)
```

will display: {'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}



```

1 states = {}
2 states.update({"Haryana": "Chandigarh"})
3 states.update({"Bihar": "Patna"})
4 states.update({"west bengal" : "kolkata"})
5 print( states)
6
Run main
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
{'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
Process finished with exit code 0

```

If a dictionary is used as the sequence in a for statement, it traverses the keys of the dictionary. This loop prints each key and the corresponding value:

```
for key in states :
```

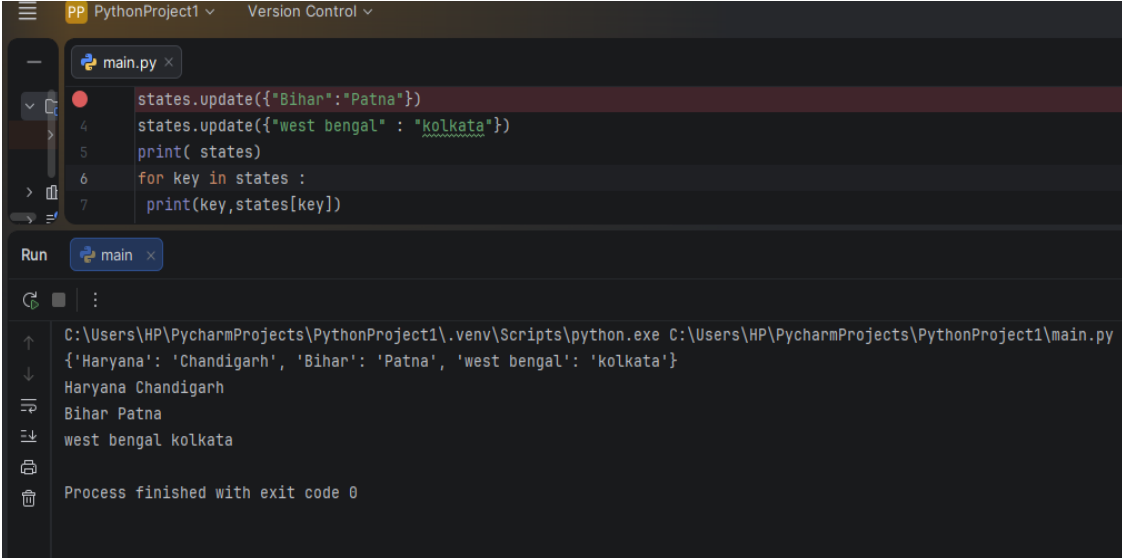
```
print(key, states[key])
```

will display:

```

Haryana Chandigarh
Bihar Patna
west bengal kolkata

```



```
main.py
1 states.update({"Bihar": "Patna"})
2 states.update({"west bengal": "kolkata"})
3 print( states)
4
5 for key in states :
6     print(key,states[key])
```

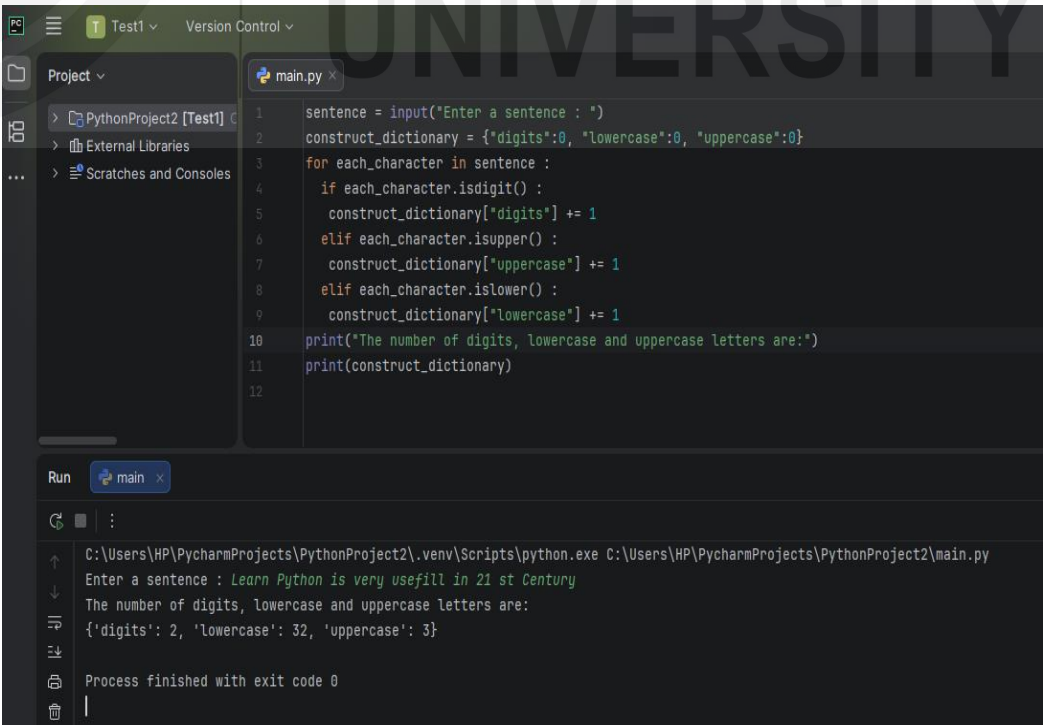
Run main

```
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
{'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
Haryana Chandigarh
Bihar Patna
west bengal kolkata
Process finished with exit code 0
```

Python Program which accepts a sentence and calculate the number of digits, uppercase laetters and owercase letters using dictionary:

```
sentence = input("Enter a sentence : ")
construct_dictionary = {"digits":0, "lowercase":0, "uppercase":0}
for each_character in sentence :
    if each_character.isdigit() :
        construct_dictionary["digits"] += 1
    elif each_character.isupper() :
        construct_dictionary["uppercase"] += 1
    elif each_character.islower() :
        construct_dictionary["lowercase"] += 1
print("The number of digits, lowercase and uppercase letters are:")
print(construct_dictionary)
```

Fig. 4: Screen Shot of execution of above Python program



```
main.py
1 sentence = input("Enter a sentence : ")
2 construct_dictionary = {"digits":0, "lowercase":0, "uppercase":0}
3 for each_character in sentence :
4     if each_character.isdigit() :
5         construct_dictionary["digits"] += 1
6     elif each_character.isupper() :
7         construct_dictionary["uppercase"] += 1
8     elif each_character.islower() :
9         construct_dictionary["lowercase"] += 1
10 print("The number of digits, lowercase and uppercase letters are:")
11 print(construct_dictionary)
```

Run main

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\main.py
Enter a sentence : Learn Python is very usefill in 21 st Century
The number of digits, lowercase and uppercase letters are:
{'digits': 2, 'lowercase': 32, 'uppercase': 3}
Process finished with exit code 0
```

It is very important to note in the above program that proper Python code indentation is necessary. You have to remember that in Python, indentation is a strict syntactical requirement used to define the structure and scope of code blocks. Other programming languages like Java, C, or C++ use curly braces {} to group statements; Python relies entirely on whitespace at the beginning of a line to determine which commands belong together.

Dictionary **del** statement :

To delete the **key:value pair**, use the **del** statement followed by the name of the dictionary along with the key you want to delete.

if

```
states contains: {'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
```

then after

```
del states["west bengal"]
```

```
states will contain: {'Haryana': 'Chandigarh', 'Bihar' : 'Patna'}
```

2.7.3.3 Relation between Tuples and Dictionaries

In Python, tuples and dictionaries are two closely interconnected data structures. Although they have completely different underlying architectures, they still frequently work together to convert, index, and unpack data. You can use tuples as key:value pairs to build dictionaries.

If tuple pm_year =

```
(('jln',1947),('lbs',1964),('ig',1966),('rg',1984),('pvn',1991),('abv',1998),('nm',2014))
```

Then

print(pm_year) will display:

```
(('jln', 1947),
 ('lbs', 1964),
 ('ig', 1966),
 ('rg', 1984),
 ('pvn', 1991),
 ('abv', 1998),
 ('nm', 2014))
```

If you construct dictionary using tuple pm_year:

```
pm_year_dict = dict(pm_year)
```

then

print(pm_year_dict) will display:

```
{'jln': 1947,
 'lbs': 1964,
 'ig': 1966,
```

```
'rg': 1984,  
'pvn': 1991,  
'abv': 1998,  
'nm': 2014}
```

You can convert tuples to dictionaries by passing the tuple name to the **dict()** function. This is achieved by nesting tuples within tuples, where each nested tuple has two items. The first item becomes the key and the second item becomes its value when the tuple gets converted to a dictionary.



The screenshot shows a Python IDE with a project named 'PythonProject2 [Test1]'. The main.py file contains the following code:

```
12  
13 pm_year = (('jln', 1947), ('lbs', 1964), ('ig', 1966), ('rg', 1984), ('pvn', 1991), ('abv', 1998), ('nm', 2014))  
14 print("tuple is:")  
15 print( pm_year )  
16 pm_year_dict = dict(pm_year)  
17 print("dictionary is:")  
18 print( pm_year_dict )  
19
```

The Run window shows the output of the program:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\main.py  
tuple is:  
(('jln', 1947), ('lbs', 1964), ('ig', 1966), ('rg', 1984), ('pvn', 1991), ('abv', 1998), ('nm', 2014))  
dictionary is:  
{'jln': 1947, 'lbs': 1964, 'ig': 1966, 'rg': 1984, 'pvn': 1991, 'abv': 1998, 'nm': 2014}  
Process finished with exit code 0
```

2.7.4 Sets

A **set** is an unordered collection with no duplicate items. Primary uses of sets include membership testing and eliminating duplicate entries. Sets also support mathematical operations, such as union, intersection, difference, and symmetric difference.

Curly braces{} or the set() function can be used to create sets with a comma-separated list of items inside curly brackets{}.

Note: In Python to create an empty set you have to use set() and not{} as the latter creates an empty dictionary.

Sets are mutable and indexing is not possible in sets, since set items are unordered. You cannot access or change a set item using indexing or slicing.

Example:

```
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
```

When you print basket, you will get:

```
{'apple', 'banana', 'orange', 'pear'}
```

You may notice that a set is a collection of unique items. Duplicate items are removed from the set basket. Here, the set will contain only one item of 'orange' and 'apple'.

You can check whether an item is in a set or not using the **in keyword**.

'orange' in basket will give **True** and 'grapes' in basket will result to **False**

The built-in function len () gives the total number of elements in a set.

len(basket) will return 4

Set elements sorting:

As sets are inherently unordered collections of unique elements, Python set objects do not have a built-in .sort() method to sort the elements of a set. If you want to sort set elements, you can do it by using the global sorted() function, which accepts the set as an argument and returns a new, sorted list containing the set's elements.

sorted(basket) will produce ['apple', 'banana', 'orange', 'pear']

Some examples:

```
a = set('abrakadabra')
```

then print (a) will display: {'a', 'b', 'd', 'k', 'r'}

```
b = set('alexander')
```

theb print(b) will display: {'a', 'd', 'e', 'l', 'n', 'r', 'x'}

```
a-b          #letters present in set a, but not in set b
```

will result to: {'b', 'k'}

```
a | b          #Letters present in set a, set b, or both
```

will result to: {'a', 'b', 'd', 'e', 'k', 'l', 'n', 'r', 'x'}

```
a & b          #letters present in both set a and set b
```

will result to: {'a', 'd', 'r'} and

```
a ^ b          #letters present in set a or set b, but not both
```

will result to: {'b', 'e', 'k', 'l', 'n', 'x'}

2.7.4.1 Set Methods

You can get a list of all methods associated with a set by passing the set to dir().

Table -18 lists the various methods associated with the set.

Table -18: Various Set Methods

Set Methods	Syntax	Description
add()	set_name.add(item)	The add() method adds an item to the set set_name.
clear()	set_name.clear()	The clear() method removes all the items from the set set_name.
difference()	set_name. difference(*others)	The difference() method returns a new set with items in the set set_name that are not

		in the other sets.
discard()	set_name. discard(item)	The discard() method removes an item from the set set_name if it is present.
intersection()	set_name. intersection(*others)	The intersection() method returns a new set with items common to the set set_name and all other sets.
isdisjoint()	set_name. isdisjoint(other)	The isdisjoint() method returns True if the set set_name has no items in common with the other set. Sets are disjoint if and only if their intersection is the empty set.
issubset()	set_name. issubset(other)	The issubset() method returns True if every item in the set set_name is in the other set.
issuperset()	set_name. issuperset(other)	The issuperset() method returns True if every element in the other set is in the set set_name.
pop()	set_name.pop()	The method pop() removes and returns an arbitrary item from the set set_name. It raises KeyError if the set is empty.
remove()	set_name. remove(item)	The method remove() removes an item from the set set_name. It raises KeyError if the item is not contained in the set.
symmetric_difference()	set_name. symmetric_difference(other)	The method symmetric_difference() returns a new set with items from the set set_name and all other sets.
union()	set_name. union(*others)	The method union() returns a new set with items from the set set_name and all other sets.
update()	set_name. update(*others)	Update the set set_name by adding items from all other sets

Note: Replace the words “set_name”, “other” and “others” mentioned in the syntax with your actual set names in your code.

Python program to demonstrate methods of set:

Program:

```
f1 = {'sunflower', 'roses', 'lavender', 'tulips', 'goldcrest'}
f2 = {'roses', 'tulips', 'lilies', 'daisies'}
f2.add('orchids')
print("Set f1:", f1)
print("Set f2:", f2)
print("Difference f2 and f1:", f2.difference(f1))
print("Intersection f2 and f1:", f2.intersection(f1))
print("Is distinct f2, f1:", f2.isdisjoint(f1))
print("Intersection f2 and f1:", f2.issuperset(f1))
print("Is subset f2 and f1:", f2.issubset(f1))
print("Symmetric Difference f2 and f1:", f2.symmetric_difference(f1))
print("Update f2:", f2.update(f1))
print("Discard f2- rose :", f2.discard("roses"))
print("Pop f1:", f1.pop())
print("Clear f2:", f2.clear())
```

Output:

Set f1: {'goldcrest', 'sunflower', 'lavender', 'tulips', 'roses'}

Set f2: {'tulips', 'roses', 'lilies', 'daisies', 'orchids'}
 Difference f2 and f1: {'lilies', 'daisies', 'orchids'}
 Intersection f2 and f1: {'tulips', 'roses'}
 Is distinct f2, f1: False
 Intersection f2 and f1: False
 Is subset f2 and f1: False
 Symmetric Difference f2 and f1: {'lilies', 'goldcrest', 'sunflower', 'lavender',
 'daisies', 'orchids'}
 Update f2: None
 Discard f2- rose : None
 Pop f1: goldcrest
 Clear f2: None

The screenshot shows a PyCharm IDE with a file named `main.py` containing the following Python code:

```

20 f1 = {'sunflower', 'roses', 'lavender', 'tulips', 'goldcrest'}
21 f2 = {'roses', 'tulips', 'lilies', 'daisies'}
22 f2.add('orchids')
23 print("Set f1:", f1)
24 print("Set f2:", f2)
25 print("Difference f2 and f1:", f2.difference(f1))
26 print("Intersection f2 and f1:", f2.intersection(f1))
27 print("Is distinct f2, f1:", f2.isdisjoint(f1))
28 print("Intersection f2 and f1:", f2.issuperset(f1))
29 print("Is subset f2 and f1:", f2.issubset(f1))
30 print("Symmetric Difference f2 and f1:", f2.symmetric_difference(f1))
31 print("Update f2:", f2.update(f1))
32 print("Discard f2- rose :", f2.discard("roses"))
33 print("Pop f1:", f1.pop())
34 print("Clear f2:", f2.clear())
35

```

The Run console at the bottom displays the output of this code:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2/main.py
Set f1: {'goldcrest', 'sunflower', 'lavender', 'tulips', 'roses'}
Set f2: {'tulips', 'roses', 'lilies', 'daisies', 'orchids'}
Difference f2 and f1: {'lilies', 'daisies', 'orchids'}
Intersection f2 and f1: {'tulips', 'roses'}
Is distinct f2, f1: False
Intersection f2 and f1: False
Is subset f2 and f1: False
Symmetric Difference f2 and f1: {'lilies', 'goldcrest', 'sunflower', 'lavender', 'daisies', 'orchids'}
Update f2: None
Discard f2- rose : None
Pop f1: goldcrest
Clear f2: None

```

2.7.4.2 Traversing of Sets

One can iterate through each item in a set using a for loop. Example code snippet for such operation is as follows:

for flower in f1 :

 print(flower, “ is a flower”)

will result into:

 lavender is a flower

 tulips is a flower

 goldcrest is a flower

 sunflower is a flower

2.7.4.3 Frozenset

A **frozenset** is basically the same as a set, except that it is *immutable*. Once a frozenset is created, its items cannot be changed. Because they are immutable, you can use them as members of other sets and as dictionary keys. Frozen sets have the same functions as sets, except those functions which change the contents (like update, remove, etc.).

Creating a frozen set:

```
fs = frozenset(["g","o","o","d"])    #creating/declaring frozenset
print( fs)                           #displaying the contents of frozenset
will display: frozenset({'d', 'g', 'o'})
```

here frozenset is used within a set:

```
pets = set([fs, "horse", "cow"]) #Frozenset type is used within a set
print( pets)
will display: {'cow', frozenset({'d', 'g', 'o'}), 'horse'}
```

Another example:

```
lang_used = {"english":59, "french":29, "spanish":21}
print( frozenset(lang_used) ) #keys in a dictionary are returned when
                              a dictionary is passed as an argument
                              to frozenset() function.
will display: frozenset({'english', 'french', 'spanish'})
```

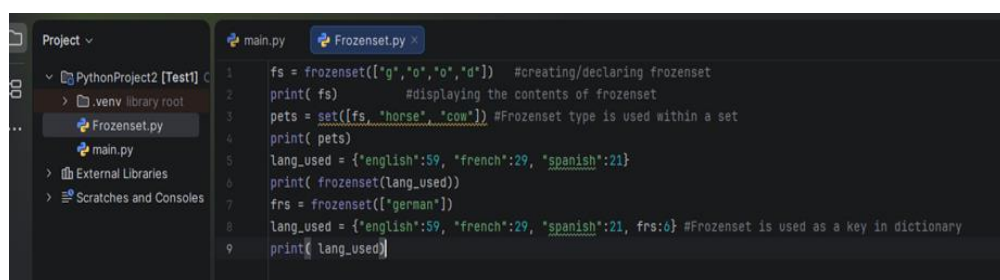
If we create frs as:

```
frs = frozenset(["german"])
#Frozenset is used as a key in dictionary
```

Then we may use frs as:

```
lang_used = {"english":59, "french":29, "spanish":21, frs:6}
print( lang_used)
will now display:
```

```
{'english': 59, 'french': 29, 'spanish': 21, frozenset({'german'}): 6}
```



```

Run FrozenSet x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\FrozenSet.py
frozenset({'o', 'd', 'g'})
{'horse', 'cow', frozenset({'o', 'd', 'g'})}
frozenset({'spanish', 'english', 'french'})
{'english': 59, 'french': 29, 'spanish': 21, frozenset({'german'})}: 6}

Process finished with exit code 0

```

Example: Use of Frozen Set

```

# Define sets of permissions as frozensets
admin_perms = frozenset(["read", "write", "delete", "manage_users"])
editor_perms = frozenset(["read", "write"])
viewer_perms = frozenset(["read"])
# Use frozensets as keys in a dictionary mapping to Role Names
role_lookup = { admin_perms: "Administrator", editor_perms: "Editor",
viewer_perms: "Viewer" }
# Simulate checking a user's permissions
current_user_perms = frozenset(["read", "write"])
# Lookup the role directly
role = role_lookup.get(current_user_perms, "Editor")
print(f"User Permissions: {set(current_user_perms)}")
print(f"Assigned Role: {role}")
# Immutability check by trying to modify a frozenset raises an error
try: editor_perms.add("delete")
except AttributeError as e: print(f"\nError Attempted: {e}")

```

```

Project v
PythonProject2 [Test1] C
  .venv library root
  FrozenSet.py
  main.py
  UseFrozenSet.py
External Libraries
Scratches and Consoles

main.py UseFrozenSet.py FrozenSet.py
1
2 # Define sets of permissions as frozensets
3 admin_perms = frozenset(["read", "write", "delete", "manage_users"])
4 editor_perms = frozenset(["read", "write"])
5 viewer_perms = frozenset(["read"])
6 # Use frozensets as keys in a dictionary mapping to Role Names
7 role_lookup = { admin_perms: "Administrator", editor_perms: "Editor", viewer_perms: "Viewer" }
8 # Simulate checking a user's permissions
9 current_user_perms = frozenset(["read", "write"])
10 # Lookup the role directly
11 role = role_lookup.get(current_user_perms, default: "Editor")
12 print(f"User Permissions: {set(current_user_perms)}")
13 print(f"Assigned Role: {role}")
14 # Immutability check by trying to modify a frozenset raises an error
15 try: editor_perms.add("delete")
16 except AttributeError as e: print(f"\nError Attempted: {e}")
17

Run UseFrozenSet x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\UseFrozenSet.py
User Permissions: {'read', 'write'}
Assigned Role: Editor

Error Attempted: 'frozenset' object has no attribute 'add'

Process finished with exit code 0

```

The program above demonstrates how to use a frozenset. This program also uses exception handling, which we will discuss in detail in a unit of the next block of this course.

Check your progress – 3

1. What will be the output of the following Python code?

```
d1={"Ravi":5,"Mohan":6,"Sohan":7}
```

```
print(d1[0])
```

- a) Ravi
- b) 5
- c) {"Ravi":5}
- d) Error

2. What is the output of the following union operation?

```
set1 = {10, 20, 30, 40}
```

```
set2 = {50, 20, "10", 60}
```

```
set3 = set1.union (set2)
```

```
print(set3)
```

- a) {40, 10, 50, 20, 60, 30}
- b) {40, '10', 50, 20, 60, 30}
- c) {40, 10, '10', 50, 20, 60, 30}
- d) SyntaxError: Different types cannot be used with sets

3. What is the output of the following set operation

```
set1 = {"Yellow", "Orange", "Black"}
```

```
set1.discard("Blue")
```

```
print(set1)
```

- a) {'Yellow', 'Orange', 'Black'}
- b) {'Yellow', 'Orange'}
- c) {'Yellow'}
- d) KeyError: 'Blue'

4. What is the difference between a set and a dictionary?

.....

.....

5. What is the difference between remove() and discard() in sets?

.....

.....

2.8 SUMMARY

This unit explained the basics of Python programming, including identifiers and variables. This unit also covered statements and expressions, operators, lists, tuples, sets, and dictionaries, which are the backbone of Python and make it unique and easy to program. These basic structures can be combined with control flow statements, which ultimately form a Python program. Mastering these core elements lays a solid foundation for writing modular, efficient, and readable code. This unit also sets the stage for developing reusable functions and leveraging Python's rich ecosystem of built-in and third-party libraries.

2.9 ANSWER TO CHECK YOUR PROGRESS

Answer to check your progress-1

1. d
2. d
3. b
4. b
5. d
6. b
7. a
8. c

9.

Identifiers must follow these rules:

- Must start with a letter (a–z or A–Z) or underscore (_)
- Cannot start with a digit
- Can contain letters, digits, and underscores
- Cannot be a Python keyword
- Case-sensitive

10 .

Yes, expressions are often part of statements.

`X = a+b`

Here, `a + b` is an expression inside an assignment statement.

Explanation:

`x = 10 + 5`

The Expression: `10 + 5`

- Python first evaluates this expression to produce the value 15.

The Statement: `x = 10 + 5`

- The entire line is an assignment statement. It takes the evaluated value (15) and assigns it to the variable `x`.

11.

No, Python does not require explicit variable declaration. However, it is a must to assign a value to a variable before you try to read or use it.

12.

`/` performs floating-point division and `//` performs floor division

In Python, the key difference between “`/`” and “`//`” is how they handle fractional parts (decimals) in division:

`/` (Floating-Point / True Division): Divides two numbers and always returns a float, keeping the exact decimal value.

Example:

`print(10 / 3)`

Output: 3.3333333333333335

`print(16 / 2)` # Output: 8.0 (Notice it is 8.0, not 5)

- `//` (Floor Division): Divides two numbers and rounds down to the nearest whole number (towards negative infinity), discarding the remainder.

Example:

`print(10 // 3)`

Output: 3

`print(10 // 2)`

Output: 5

Important to note that if either number is a float, in // operation, the result is returned as a float:

```
print(10.0 // 3), output will be : 3.0
```

Answer to check your progress-2

1. c
2. b
3. c
4. A list is said to be mutable because its elements can be changed, added, or removed after the list has been created, without creating a new list.
5. The “in” and “not in” operators in Python are membership operators used with lists to check whether an element is present or absent.
6. Tuples themselves are immutable, meaning their structure (size and element positions) cannot change, but the objects stored inside a tuple can be mutable. Python allows this because immutability applies to the tuple container, not to the contents it references.
7.

```
list_of_tuples = [(1, 2), (3, 4), (5, 6)]  
print(list_of_tuples)
```

Output:

```
[(1, 2), (3, 4), (5, 6)]
```

Answer to Check your progress-3

1. d
2. c
3. a
- 4.

Aspect	Set	Dictionary
Definition	A collection of unique elements	A collection of key–value pairs
Data stored	Only values	Keys mapped to values
Uniqueness	All elements must be unique	Keys must be unique; values may repeat
Syntax	{1, 2, 3}	{"a": 1, "b": 2}
Access	Elements are not accessed by index or key	Values are accessed using keys
Mutability	Mutable (elements can be added/removed)	Mutable (key–value pairs can be added/updated/removed)
Use case	Membership testing, removing duplicates	Storing and retrieving data with meaningful keys

5. In Python sets, both `remove()` and `discard()` are used to delete elements.

i. `remove()`

- Removes the specified element from the set.
- Raises a `KeyError` if the element is not found.

```
my_set = {1, 2, 3}  
my_set.remove(2) # Removes 2  
print(my_set)    # {1, 3}  
my_set.remove(5) # KeyError: 5
```

ii. discard()

- Removes the specified element from the set.
 - Does NOT raise an error if the element is not found; it does nothing.
- ```
my_set = {1, 2, 3}
my_set.discard(2) # Removes 2
print(my_set) # {1, 3}
my_set.discard(5) # No error, does nothing
print(my_set) # {1, 3}
```

---

## 2.10 REFERENCES/FURTHER READINGS

---

1. Yashavant Kanetkar and Aditya Kanetkar, “Let Us Python”, 4<sup>th</sup> Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , “Python For Programmers” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, “Python Programming a Modular Approach” , 1<sup>st</sup> Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , “Python Programming” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , “Problem Solving and Python Programming”, 1<sup>st</sup> Edition, Ane Books , New Delhi, 2018

THE PEOPLE'S  
UNIVERSITY